Chapter 2. RAG Part I: Indexing Your Data

In the previous chapter, you learned about the important building blocks used to create an LLM application using LangChain. You also built a simple AI chatbot consisting of a prompt sent to the model and the output generated by the model. But there are major limitations to this simple chatbot.

What if your use case requires knowledge that the model wasn't trained on? For example, let's say you want to use AI to ask questions about a company, but the information is contained in a private PDF or other type of document. While we've seen model providers enriching their training datasets to include more and more of the world's public information (no matter what format it is stored in), two major limitations continue to exist in LLM's knowledge corpus:

Private data

Information that isn't publicly available is, by definition, not included in the training data of LLMs.

Current events

Training an LLM is a costly and time-consuming process that can span multiple years, with data-gathering being one of the first steps. This results in what is called the knowledge cutoff, or a date beyond which the LLM has no knowledge of real-world events; usually this would be the date the training set was finalized. This can be anywhere from a few months to a few years into the past, depending on the model in question.

In either case, the model will most likely hallucinate (find misleading or false information) and respond with inaccurate information. Adapting the prompt won't resolve the issue either because it relies on the model's current knowledge.

The Goal: Picking Relevant Context for

LLMs

If the only private/current data you needed for your LLM use case was one to two pages of text, this chapter would be a lot shorter: all you'd need to make that information available to the LLM is to include that entire text in every single prompt you sent to the model.

The challenge in making data available to LLMs is first and foremost a quantity problem. You have more information than can fit in each prompt you send to the LLM. Which small subset of your large collection of text do you include each time you call the model? Or in other words, how do you pick (with the aid of the model) which text is most relevant to answer each question?

In this chapter and the next, you'll learn how to overcome this challenge in two steps:

1. *Indexing* your documents, that is, preprocessing them in a way where your application can easily find the most relevant ones for each question
2. *Retrieving* this external data from the index and using it as *context* for the LLM to generate an accurate output based on your data

This chapter focuses on indexing, the first step, which involves preprocessing your documents into a format that can be understood and searched with LLMs. This technique is called *retrieval-augmented generation* (RAG). But before we begin, let's discuss *why* your documents require preprocessing.

Let's assume you would like to use LLMs to analyze the financial performance and risks in Tesla's 2022 annual report, which is stored as text in PDF format. Your goal is to be able to ask a question like "What key risks did Tesla face in 2022?" and get a humanlike response based on context from the risk factors section of the document.

Breaking it down, there are four key steps (shown in Figure 2-1) that you'd need to take in order to achieve this goal:

1. Extract the text from the document.
2. Split the text into manageable chunks.
3. Convert the text into numbers that computers can understand.
4. Store these number representations of your text somewhere that makes it easy and fast to retrieve the relevant sections of your document to answer a given question.

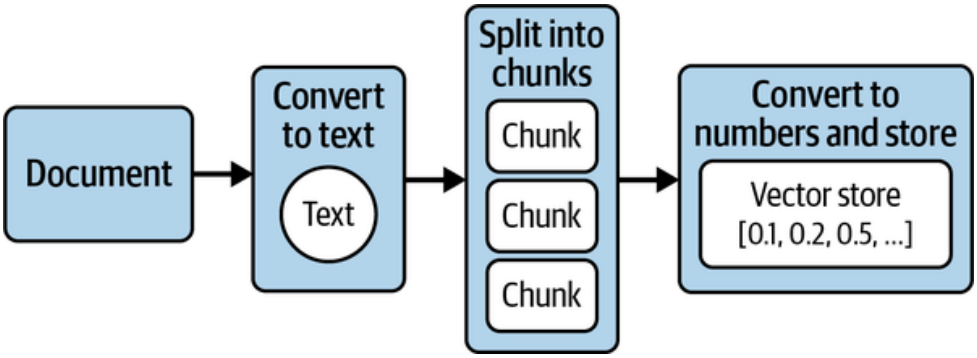

Figure 2-1. Four key steps to preprocess your documents for LLM usage

Figure 2-1 illustrates the flow of this preprocessing and transformation of your documents, a process known as ingestion. *Ingestion* is simply the process of converting your documents into numbers that computers can understand and analyze, and storing them in a special type of database for efficient retrieval. These numbers are formally known as *embeddings*, and this special type of database is known as a *vector store*. Let's look a little more closely at what embeddings are and why they're important, starting with something simpler than LLM-powered embeddings.

Embeddings: Converting Text to Numbers

Embedding refers to representing text as a (long) sequence of numbers. This is a lossy representation—that is, you can't recover the original text from these number sequences, so you usually store both the original text and this numeric representation.

So, why bother? Because you gain the flexibility and power that comes with working with numbers: you can do math on words! Let's see why that's exciting.

Embeddings Before LLMs

Long before LLMs, computer scientists were using embeddings—for instance, to enable full-text search capabilities in websites or to classify emails as spam. Let's see an example:

1. Take these three sentences:
   - What a sunny day.
   - Such bright skies today.
   - I haven't seen a sunny day in weeks.
2. List all unique words in them: *what*, *a*, *sunny*, *day*, *such*, *bright*, and so on.
3. For each sentence, go word by word and assign the number 0 if not present, 1 if used once in the sentence, 2 if present twice, and so on.

Table 2-1 shows the result.

Table 2-1. Word embeddings for three sentences

Word	What a sunny day.	Such bright skies today.	I haven't seen a sunny day in weeks.
what	1	0	0
a	1	0	1
sunny	1	0	1
day	1	0	1
such	0	1	0
bright	0	1	0
skies	0	1	0
today	0	1	0
I	0	0	1
haven't	0	0	1
seen	0	0	1
in	0	0	1
weeks	0	0	1

In this model, the embedding for *I haven't seen a sunny day in weeks* is the sequence of numbers *0 1 1 1 0 0 0 0 1 1 1 1 1*. This is called the *bag-of-words* model, and these embeddings are also called *sparse embeddings* (or sparse vectors—*vector* is another word for a sequence of numbers), because a lot of the numbers will be 0. Most English sentences use only a very small subset of all existing English words.

You can successfully use this model for:

Keyword search

You can find which documents contain a given word or words.

Classification of documents

You can calculate embeddings for a collection of examples previously labeled as email spam or not spam, average them out, and obtain average word frequencies for each of the classes (spam or not spam). Then, each new document is compared to those averages and classified accordingly.

The limitation here is that the model has no awareness of meaning, only of the actual words used. For instance, the embeddings for *sunny day* and *bright skies* look very different. In fact they have no words in common, even though we know they have similar meaning. Or, in the email classification problem, a would-be spammer can trick the filter by replacing common "spam words" with their synonyms.

In the next section, we'll see how semantic embeddings address this limitation by using numbers to represent the meaning of the text, instead of the exact words found in the text.

LLM-Based Embeddings

We're going to skip over all the ML developments that came in between and jump straight to LLM-based embeddings. Just know that there was a gradual evolution from the simple method outlined in the previous section to the sophisticated method described in this one.

You can think of embedding models as an offshoot from the training process of LLMs. If you remember from the Preface, the LLM training process (learning from vast amounts of written text) enables LLMs to complete a prompt (or input) with the most appropriate continuation (output). This capability stems from an understanding of the meaning of words and sentences in the context of the surrounding text, learned from how words are used together in the training texts. This *understanding* of the meaning (or semantics) of the prompt can be extracted as a numeric representation (or embedding) of the input text, and can be used directly for some very interesting use cases too.

In practice, most embedding models are trained for that purpose alone, following somewhat similar architectures and training processes as LLMs, as that is more efficient and results in higher-quality embeddings.[1]

An *embedding model* then is an algorithm that takes a piece of text and outputs a numerical representation of its meaning—technically, a long list of floating-point (decimal) numbers, usually somewhere between 100 and 2,000 numbers, or *dimensions*. These are also called *dense* embeddings, as opposed to the *sparse* embeddings of the previous section, as here usually all dimensions are different from 0.

TIP

Different models produce different numbers and different sizes of lists. All of these are specific to each model; that is, even if the size of the lists matches, you cannot compare embeddings from different models. Combining embeddings from different models should always be avoided.

Semantic Embeddings Explained

Consider these three words: *lion*, *pet*, and *dog*. Intuitively, which pair of these words share similar characteristics to each other at first glance? The obvious answer is *pet* and *dog*. But computers do not have the ability to tap into this intuition or nuanced understanding of the English language. In order for a computer to differentiate between a lion, pet, or dog, you need to be able to translate them into the language of computers, which is numbers.

Figure 2-2 illustrates converting each word into hypothetical number representations that retain their meaning.

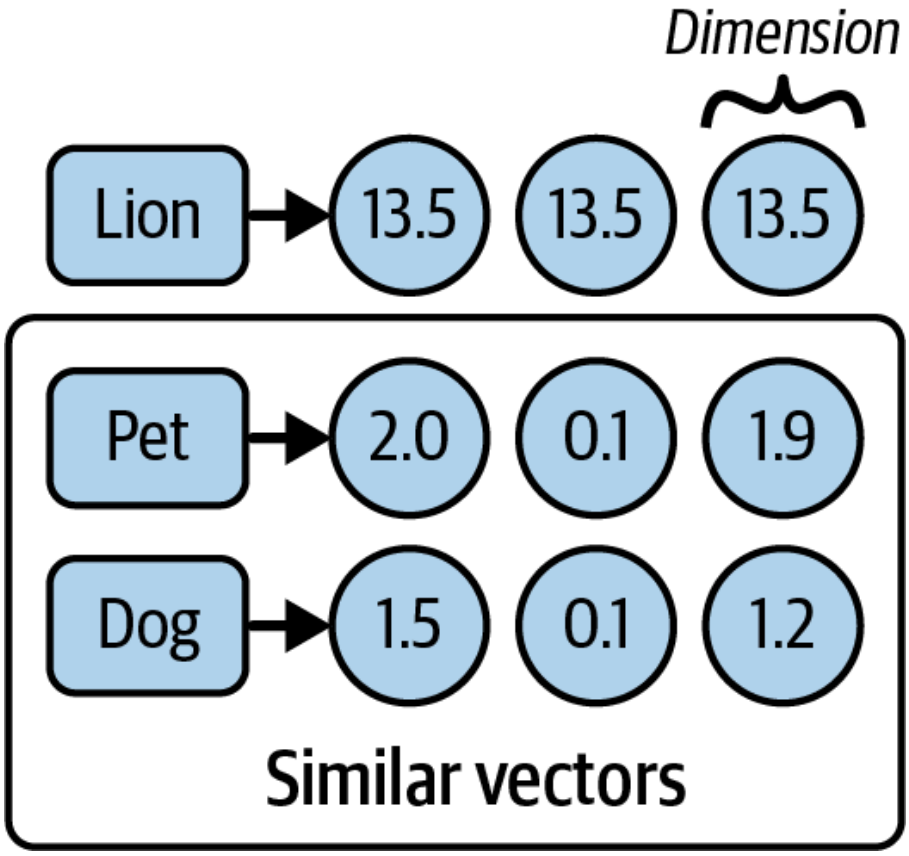

Figure 2-2. Semantic representations of words

Figure 2-2 shows each word alongside its corresponding semantic embedding. Note that the numbers themselves have no particular meaning, but instead the sequences of numbers for two words (or sentences) that are close in meaning should be *closer* than those of unrelated words. As you can see, each number is a *floating-point value*, and each of them represents a semantic *dimension*. Let's see what we mean by *closer*:

If we plot these vectors in a three-dimensional space, it could look like Figure 2-3.

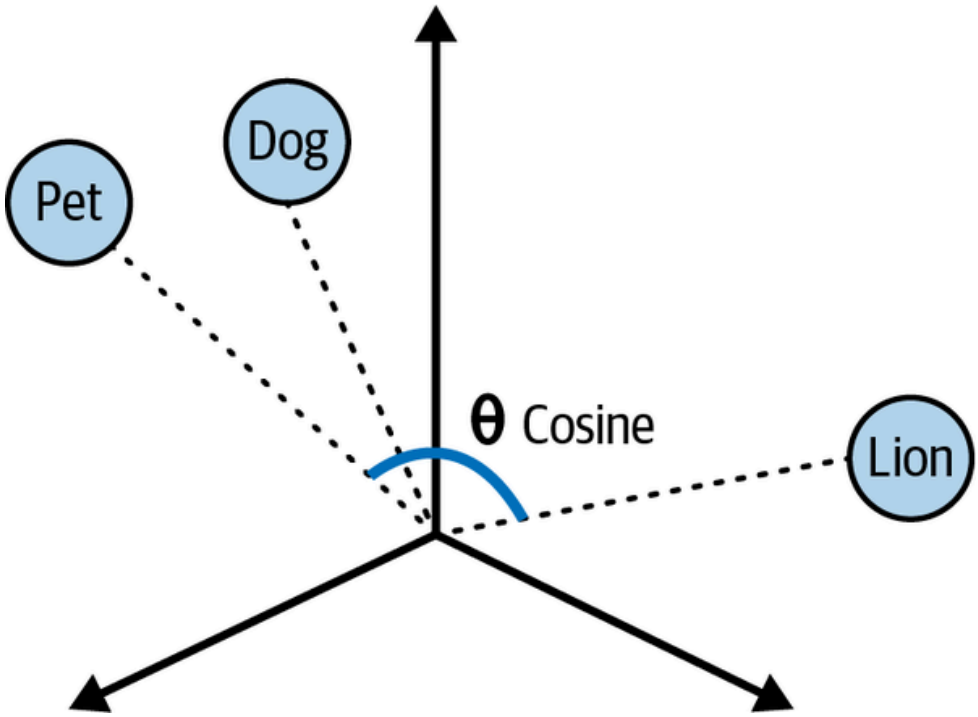

Figure 2-3. Plot of word vectors in a multidimensional space

Figure 2-3 shows the *pet* and *dog* vectors are closer to each other in distance than the *lion* plot. We can also observe that the angles between each plot varies depending on how similar they are. For example, the words *pet* and *lion* have a wider angle between one another than the *pet* and *dog* do, indicating more similarities shared by the latter word pairs. The narrower the angle or shorter the distance between two vectors, the closer their similarities.

One effective way to calculate the degree of similarity between two vectors in a multidimensional space is called cosine similarity. *Cosine similarity* computes the dot product of vectors and divides it by the product of their magnitudes to output a number between –1 and 1, where 0 means the vectors share no correlation, –1 means they are absolutely dissimilar, and 1 means they are absolutely similar. So, in the case of our three words here, the cosine similarity between *pet* and *dog* could be 0.75, but between *pet* and *lion* it might be 0.1.

The ability to convert sentences into embeddings that capture semantic meaning and then perform calculations to find semantic similarities between different sentences enables us to get an LLM to find the most relevant documents to answer questions about a large body of text like our Tesla PDF document. Now that you understand the big picture, let's revisit the first step (indexing) of preprocessing your document.

OTHER USES FOR EMBEDDINGS

These sequences of numbers and vectors have a number of interesting properties:

- As you learned earlier, if you think of a vector as describing a point in high-dimensional space, points that are closer together have more similar meanings, so a distance function can be used to measure similarity.
- Groups of points close together can be said to be related; therefore, a clustering algorithm can be used to identify topics (or clusters of points) and classify new inputs into one of those topics.
- If you average out multiple embeddings, the average embedding can be said to represent the overall meaning of that group; that is, you can embed a long document (for instance, this book) by:
  1. Embedding each page separately
  2. Taking the average of the embeddings of all pages as the book embedding
- You can "travel" the "meaning" space by using the elementary math operations of addition and subtraction: for instance, the operation *king – man + woman = queen*. If you take the meaning (or semantic embedding) of *king*, subtract the meaning of *man*, presumably you arrive at the more abstract meaning of *monarch*, at which point, if you add the meaning of *woman*, you've arrived close to the meaning (or embedding) of the word *queen*.
- There are models that can produce embeddings for nontext content, for instance, images, videos, and sounds, in addition to text. This enables, for instance, finding images that are most similar or relevant for a given sentence.

We won't explore all of these attributes in this book, but it's useful to know they can be used for a number of applications such as:

Search

Finding the most relevant documents for a new query

Clustering

Given a body of documents, dividing them into groups (for instance, topics)

Classification

Assigning a new document to a previously identified group or label (for instance, a topic)

Recommendation

Given a document, surfacing similar documents

Detecting anomalies

> Identifying documents that are very dissimilar from previously seen ones

We hope this leaves you with some intuition that embeddings are quite versatile and can be put to good use in your future projects.

Converting Your Documents into Text

As mentioned at the beginning of the chapter, the first step in preprocessing your document is to convert it to text. In order to achieve this, you would need to build logic to parse and extract the document with minimal loss of quality. Fortunately, LangChain provides *document loaders* that handle the parsing logic and enable you to "load" data from various sources into a `Document` class that consists of text and associated metadata.

For example, consider a simple *.txt* file. You can simply import a LangChain `TextLoader` class to extract the text, like this:

Python

```
from langchain_community.document_loaders import TextLoader

loader = TextLoader("./test.txt")
loader.load()
```

JavaScript

```
import { TextLoader } from "langchain/document_loaders/fs/text";

const loader = new TextLoader("./test.txt");

const docs = await loader.load();
```

The output:

```
[Document(page_content='text content \n', metadata={'line_number': 0, 'sou
    './test.txt'})]
```

The previous code block assumes that you have a file named `test.txt` in your current directory. Usage of all LangChain document loaders follows a similar pattern:

1. Start by picking the loader for your type of document from the long list of integrations.
2. Create an instance of the loader in question, along with any parameters to configure it, including the location of your documents (usually a filesystem path or web address).
3. Load the documents by calling `load()`, which returns a list of documents ready to pass to the next stage (more on that soon).

Aside from *.txt* files, LangChain provides document loaders for other popular file types including *.csv*, *.json*, and Markdown, alongside integrations with popular platforms such as Slack and Notion.

For example, you can use `WebBaseLoader` to load HTML from web URLs and parse it to text.

Install the beautifulsoup4 package:

```
pip install beautifulsoup4
```

Python

```python
from langchain_community.document_loaders import WebBaseLoader

loader = WebBaseLoader("https://www.langchain.com/")
loader.load()
```

JavaScript

```javascript
// install cheerio: npm install cheerio
import {
  CheerioWebBaseLoader
} from "@langchain/community/document_loaders/web/cheerio";

const loader = new CheerioWebBaseLoader("https://www.langchain.com/");

const docs = await loader.load();
```

In the case of our Tesla PDF use case, we can utilize LangChain's `PDFLoader` to extract text from the PDF document:

Python

```python
# install the pdf parsing library
# pip install pypdf

from langchain_community.document_loaders import PyPDFLoader
```

```
loader = PyPDFLoader("./test.pdf")
pages = loader.load()
```

JavaScript

```
// install the pdf parsing library: npm install pdf-parse

import { PDFLoader } from "langchain/document_loaders/fs/pdf";

const loader = new PDFLoader("./test.pdf");

const docs = await loader.load();
```

The text has been extracted from the PDF document and stored in the `Document` class. But there's a problem. The loaded document is over 100,000 characters long, so it won't fit into the context window of the vast majority of LLMs or embedding models. In order to overcome this limitation, we need to split the `Document` into manageable chunks of text that we can later convert into embeddings and semantically search, bringing us to the second step (retrieving).

TIP

LLMs and embedding models are designed with a hard limit on the size of input and output tokens they can handle. This limit is usually called *context window*, and usually applies to the combination of input and output; that is, if the context window is 100 (we'll talk about units in a second), and your input measures 90, the output can be at most of length 10. Context windows are usually measured in number of tokens, for instance 8,192 tokens. Tokens, as mentioned in the Preface, are a representation of text as numbers, with each token usually covering between three and four characters of English text.

Splitting Your Text into Chunks

At first glance it may seem straightforward to split a large body of text into chunks, but keeping *semantically* related (related by meaning) chunks of text together is a complex process. To make it easier to split large documents into small, but still meaningful, pieces of text, LangChain provides `RecursiveCharacterTextSplitter`, which does the following:

1. Take a list of separators, in order of importance. By default these are:
    a. The paragraph separator: `\n\n`
    b. The line separator: `\n`

    c. The word separator: space character
2. To respect the given chunk size, for instance, 1,000 characters, start by splitting up paragraphs.
3. For any paragraph longer than the desired chunk size, split by the next separator: lines. Continue until all chunks are smaller than the desired length, or there are no additional separators to try.
4. Emit each chunk as a `Document`, with the metadata of the original document passed in and additional information about the position in the original document.

Let's see an example:

Python

```python
from langchain_text_splitters import RecursiveCharacterTextSplitter

loader = TextLoader("./test.txt") # or any other loader
docs = loader.load()

splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200,
)
splitted_docs = splitter.split_documents(docs)
```

JavaScript

```javascript
import { TextLoader } from "langchain/document_loaders/fs/text";
import { RecursiveCharacterTextSplitter } from "@langchain/textsplitters";

const loader = new TextLoader("./test.txt"); // or any other loader
const docs = await loader.load();

const splitter = new RecursiveCharacterTextSplitter({
  chunkSize: 1000,
  chunkOverlap: 200,
});

const splittedDocs = await splitter.splitDocuments(docs)
```

In the preceding code, the documents created by the document loader are split into chunks of 1,000 characters each, with some overlap between chunks of 200 characters to maintain some context. The result is also a list of documents, where each document is up to 1,000 characters in length, split along the natural divisions of written text—paragraphs, new lines and finally, words. This uses the structure of the text to keep each chunk a consistent, readable snippet of text.

`RecursiveCharacterTextSplitter` can also be used to split code languages and Markdown into semantic chunks. This is done by using keywords specific to each language as the separators, which ensures, for instance, the body of each function is kept in the same chunk, instead of split between several. Usually, as programming languages have more structure than written text, there's less need to use overlap between the chunks. LangChain contains separators for a number of popular languages, such as Python, JS, Markdown, HTML, and many more. Here's an example:

Python

```
from langchain_text_splitters import (
    Language,
    RecursiveCharacterTextSplitter,
)

PYTHON_CODE = """
def hello_world():
    print("Hello, World!")

# Call the function
hello_world()
"""
python_splitter = RecursiveCharacterTextSplitter.from_language(
    language=Language.PYTHON, chunk_size=50, chunk_overlap=0
)
python_docs = python_splitter.create_documents([PYTHON_CODE])
```

JavaScript

```
import { RecursiveCharacterTextSplitter } from "@langchain/textsplitters";

const PYTHON_CODE = `
def hello_world():
  print("Hello, World!")

# Call the function
hello_world()
`;

const pythonSplitter = RecursiveCharacterTextSplitter.fromLanguage("python
  chunkSize: 50,
  chunkOverlap: 0,
});
const pythonDocs = await pythonSplitter.createDocuments([PYTHON_CODE]);
```

The output:

```
[Document(page_content='def hello_world():\n    print("Hello, World!")'),
    Document(page_content='# Call the function\nhello_world()')]
```

Notice how we're still using `RecursiveCharacterTextSplitter` as before, but now we're creating an instance of it for a specific language, using the `from_language` method. This one accepts the name of the language, and the usual parameters for chunk size, and so on. Also notice we are now using the method `create_documents`, which accepts a list of strings, rather than the list of documents we had before. This method is useful when the text you want to split doesn't come from a document loader, so you have only the raw text strings.

You can also use the optional second argument to `create_documents` in order to pass a list of metadata to associate with each text string. This metadata list should have the same length as the list of strings and will be used to populate the metadata field of each `Document` returned.

Let's see an example for Markdown text, using the metadata argument as well:

Python

```
markdown_text = """
# LangChain

⚡ Building applications with LLMs through composability ⚡

## Quick Install

```bash
pip install langchain
```

As an open source project in a rapidly developing field, we are extremely o
    to contributions.
"""

md_splitter = RecursiveCharacterTextSplitter.from_language(
    language=Language.MARKDOWN, chunk_size=60, chunk_overlap=0
)
md_docs = md_splitter.create_documents([markdown_text],
    [{"source": "https://www.langchain.com"}])
```

JavaScript

```
const markdownText = `
# LangChain
```

```
⚡ Building applications with LLMs through composability ⚡

## Quick Install

\`\`\`bash
pip install langchain
\`\`\`

As an open source project in a rapidly developing field, we are extremely
  open to contributions.
`;

const mdSplitter = RecursiveCharacterTextSplitter.fromLanguage("markdown",
  chunkSize: 60,
  chunkOverlap: 0,
});
const mdDocs = await mdSplitter.createDocuments([markdownText],
  [{"source": "https://www.langchain.com"}]);
```

The output:

```
[Document(page_content='# LangChain',
    metadata={"source": "https://www.langchain.com"}),
 Document(page_content='⚡ Building applications with LLMs through composabi
    ⚡', metadata={"source": "https://www.langchain.com"}),
 Document(page_content='## Quick Install\n\n```bash',
    metadata={"source": "https://www.langchain.com"}),
 Document(page_content='pip install langchain',
    metadata={"source": "https://www.langchain.com"}),
 Document(page_content='```', metadata={"source": "https://www.langchain.co
 Document(page_content='As an open source project in a rapidly developing
    we', metadata={"source": "https://www.langchain.com"}),
 Document(page_content='are extremely open to contributions.',
    metadata={"source": "https://www.langchain.com"})]
```

Notice two things:

- The text is split along the natural stopping points in the Markdown document; for instance, the heading goes into one chunk, the line of text under it in a separate chunk, and so on.
- The metadata we passed in the second argument is attached to each resulting document, which allows you to track, for instance, where the document came from and where you can go to see the original.

Generating Text Embeddings

LangChain also has an `Embeddings` class designed to interface with text embedding models—including OpenAI, Cohere, and Hugging Face—and

generate vector representations of text. This class provides two methods: one for embedding documents and one for embedding a query. The former takes a list of text strings as input, while the latter takes a single text string.

Here's an example of embedding a document using OpenAI's embedding model:

Python

```python
from langchain_openai import OpenAIEmbeddings

model = OpenAIEmbeddings()

embeddings = model.embed_documents([
    "Hi there!",
    "Oh, hello!",
    "What's your name?",
    "My friends call me World",
    "Hello World!"
])
```

JavaScript

```javascript
import { OpenAIEmbeddings } from "@langchain/openai";

const model = new OpenAIEmbeddings();

const embeddings = await embeddings.embedDocuments([
  "Hi there!",
  "Oh, hello!",
  "What' s your name?",
  "My friends call me World",
  "Hello World!"
]);
```

The output:

```
[
  [
    -0.004845875, 0.004899438, -0.016358767, -0.024475135, -0.017341806,
      0.012571548, -0.019156644, 0.009036391, -0.010227379, -0.026945334,
      0.022861943, 0.010321903, -0.023479493, -0.0066544134, 0.007977734,
    0.0026371893, 0.025206111, -0.012048521, 0.012943339, 0.013094575,
    -0.010580265, -0.003509951, 0.004070787, 0.008639394, -0.020631202,
    ... 1511 more items
  ]
  [
      -0.009446913, -0.013253193, 0.013174579, 0.0057552797, -0.038993083,
```

```
      0.0077763423, -0.0260478, -0.0114384955, -0.0022683728, -0.016509168
      0.041797023, 0.01787183, 0.00552271, -0.0049789557, 0.018146982,
      -0.01542166, 0.033752076, 0.006112323, 0.023872782, -0.016535373,
      -0.006623321, 0.016116094, -0.0061090477, -0.0044155475, -0.01662709
    ... 1511 more items
  ]
  ... 3 more items
]
```

Notice that you can embed multiple documents at the same time; you should prefer this to embedding them one at a time, as it will be more efficient (due to how these models are constructed). You get back a list containing multiple lists of numbers—each inner list is a vector or embedding, as explained in an earlier section.

Now let's see an end-to-end example using the three capabilities we've seen so far:

- Document loaders, to convert any document to plain text
- Text splitters, to split each large document into many smaller ones
- Embeddings models, to create a numeric representation of the meaning of each split

Here's the code:

Python

```python
from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings

## Load the document

loader = TextLoader("./test.txt")
doc = loader.load()

"""
[
    Document(page_content='Document loaders\n\nUse document loaders to load
        from a source as `Document`\'s. A `Document` is a piece of text\nan
        associated metadata. For example, there are document loaders for
        loading a simple `.txt` file, for loading the text\ncontents of any
        page, or even for loading a transcript of a YouTube video.\n\nEvery
        document loader exposes two methods:\n1. "Load": load documents fro
        the configured source\n2. "Load and split": load documents from the
        configured source and split them using the passed in text
        splitter\n\nThey optionally implement:\n\n3. "Lazy load": load
        documents into memory lazily\n', metadata={'source': 'test.txt'})
]
"""
```

```
## Split the document

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=20,
)
chunks = text_splitter.split_documents(doc)

## Generate embeddings

embeddings_model = OpenAIEmbeddings()
embeddings = embeddings_model.embed_documents(
    [chunk.page_content for chunk in chunks]
)
"""
[[0.0053587136790156364,
 -0.0004999046213924885,
  0.038883671164512634,
 -0.003001077566295862,
 -0.00900818221271038, ...], ...]
"""
```

JavaScript

```
import { TextLoader } from "langchain/document_loaders/fs/text";
import { RecursiveCharacterTextSplitter } from "@langchain/textsplitters";
import { OpenAIEmbeddings } from "@langchain/openai";

// Load the document

const loader = new TextLoader("./test.txt");
const docs = await loader.load();

// Split the document

const splitter = new RecursiveCharacterTextSplitter({
  chunkSize: 1000,
  chunkOverlap: 200,
});
const chunks = await splitter.splitDocuments(docs)

// Generate embeddings

const model = new OpenAIEmbeddings();
await embeddings.embedDocuments(chunks.map(c => c.pageContent));
```

Once you've generated embeddings from your documents, the next step is to store them in a special database known as a vector store.

Storing Embeddings in a Vector Store

Earlier in this chapter, we discussed the cosine similarity calculation to measure the similarity between vectors in a vector space. A vector store is a database designed to store vectors and perform complex calculations, like cosine similarity, efficiently and quickly.

Unlike traditional databases that specialize in storing structured data (such as JSON documents or data conforming to the schema of a relational database), vector stores handle unstructured data, including text and images. Like traditional databases, vector stores are capable of performing create, read, update, delete (CRUD), and search operations.

Vector stores unlock a wide variety of use cases, including scalable applications that utilize AI to answer questions about large documents, as illustrated in Figure 2-4.

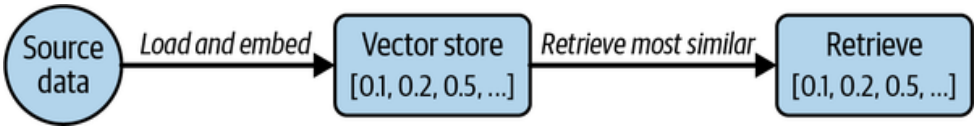

Figure 2-4. Loading, embedding, storing, and retrieving relevant docs from a vector store

Figure 2-4 illustrates how document embeddings are inserted into the vector store and how later, when a query is sent, similar embeddings are retrieved from the vector store.

Currently, there is an abundance of vector store providers to choose from, each specializing in different capabilities. Your selection should depend on the critical requirements of your application, including multitenancy, metadata filtering capabilities, performance, cost, and scalability.

Although vector stores are niche databases built to manage vector data, there are a few disadvantages working with them:

- Most vector stores are relatively new and may not stand the test of time.
- Managing and optimizing vector stores can present a relatively steep learning curve.
- Managing a separate database adds complexity to your application and may drain valuable resources.

Fortunately, vector store capabilities have recently been extended to PostgreSQL (a popular open source relational database) via the `pgvector` extension. This enables you to use the same database you're already familiar with and to power both your transactional tables (for instance your users table) as well as your vector search tables.

Getting Set Up with PGVector

To use Postgres and PGVector you'll need to follow a few setup steps:

1. Ensure you have Docker installed on your computer, following the instructions for your operating system.
2. Run the following command in your terminal; it will launch a Postgres instance in your computer running on port 6024:

```
docker run \
    --name pgvector-container \
    -e POSTGRES_USER=langchain \
    -e POSTGRES_PASSWORD=langchain \
    -e POSTGRES_DB=langchain \
    -p 6024:5432 \
    -d pgvector/pgvector:pg16
```

   Open your docker dashboard containers and you should see a green running status next to `pgvector-container`.
3. Save the connection string to use in your code; we'll need it later:

```
postgresql+psycopg://langchain:langchain@localhost:6024/langchain
```

Working with Vector Stores

Picking up where we left off in the previous section on embeddings, now let's see an example of loading, splitting, embedding, and storing a document in PGVector:

Python

```python
# first, pip install langchain-postgres
from langchain_community.document_loaders import TextLoader
from langchain_openai import OpenAIEmbeddings
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_postgres.vectorstores import PGVector
from langchain_core.documents import Document
import uuid

# Load the document, split it into chunks
raw_documents = TextLoader('./test.txt').load()
text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000,
    chunk_overlap=200)
documents = text_splitter.split_documents(raw_documents)

# embed each chunk and insert it into the vector store
embeddings_model = OpenAIEmbeddings()
```

```
connection = 'postgresql+psycopg://langchain:langchain@localhost:6024/langc
db = PGVector.from_documents(documents, embeddings_model, connection=connec
```

JavaScript

```
import { TextLoader } from "langchain/document_loaders/fs/text";
import { RecursiveCharacterTextSplitter } from "@langchain/textsplitters";
import { OpenAIEmbeddings } from "@langchain/openai";
import { PGVectorStore } from "@langchain/community/vectorstores/pgvector"
import { v4 as uuidv4 } from 'uuid';

// Load the document, split it into chunks
const loader = new TextLoader("./test.txt");
const raw_docs = await loader.load();
const splitter = new RecursiveCharacterTextSplitter({
  chunkSize: 1000,
  chunkOverlap: 200,
});
const docs = await splitter.splitDocuments(docs)

// embed each chunk and insert it into the vector store
const embeddings_model = new OpenAIEmbeddings();
const db = await PGVectorStore.fromDocuments(docs, embeddings_model, {
  postgresConnectionOptions: {
    connectionString: 'postgresql://langchain:langchain@localhost:6024/lang
  }
})
```

Notice how we reuse the code from the previous sections to first load the documents with the loader and then split them into smaller chunks. Then, we instantiate the embeddings model we want to use—in this case, OpenAI's. Note that you could use any other embeddings model supported by LangChain here.

Next, we have a new line of code, which creates a vector store given documents, the embeddings model, and a connection string. This will do a few things:

- Establish a connection to the Postgres instance running in your computer (see "Getting Set Up with PGVector".)
- Run any setup necessary, such as creating tables to hold your documents and vectors, if this is the first time you're running it.
- Create embeddings for each document you passed in, using the model you chose.
- Store the embeddings, the document's metadata, and the document's text content in Postgres, ready to be searched.

Let's see what it looks like to search documents:

Python

```
db.similarity_search("query", k=4)
```

JavaScript

```
await pgvectorStore.similaritySearch("query", 4);
```

This method will find the most relevant documents (which you previously indexed), by following this process:

- The search query—in this case, the word `query`—will be sent to the embeddings model to retrieve its embedding.
- Then, it will run a query on Postgres to find the N (in this case 4) previously stored embeddings that are most similar to your query.
- Finally, it will fetch the text content and metadata that relates to each of those embeddings.
- The model can now return a list of `Document` sorted by how similar they are to the query—the most similar first, the second most similar after, and so on.

You can also add more documents to an existing database. Let's see an example:

Python

```
ids = [str(uuid.uuid4()), str(uuid.uuid4())]
db.add_documents(
    [
        Document(
            page_content="there are cats in the pond",
            metadata={"location": "pond", "topic": "animals"},
        ),
        Document(
            page_content="ducks are also found in the pond",
            metadata={"location": "pond", "topic": "animals"},
        ),
    ],
    ids=ids,
)
```

JavaScript

```
const ids = [uuidv4(), uuidv4()];

await db.addDocuments(
```

```
  [
    {
      pageContent: "there are cats in the pond",
      metadata: {location: "pond", topic: "animals"}
    },
    {
      pageContent: "ducks are also found in the pond",
      metadata: {location: "pond", topic: "animals"}
    },
  ],
  {ids}
);
```

The `add_documents` method we're using here will follow a similar process to `fromDocuments`:

- Create embeddings for each document you passed in, using the model you chose.
- Store the embeddings, the document's metadata, and the document's text content in Postgres, ready to be searched.

In this example, we are using the optional `ids` argument to assign identifiers to each document, which allows us to update or delete them later.

Here's an example of the delete operation:

Python

```
db.delete(ids=[1])
```

JavaScript

```
await db.delete({ ids: [ids[1]] })
```

This removes the second document inserted by using its Universally Unique Identifier (UUID). Now let's see how to do this in a more systematic way.

Tracking Changes to Your Documents

One of the key challenges with working with vector stores is working with data that regularly changes, because changes mean re-indexing. And re-indexing can lead to costly recomputations of embeddings and duplications of preexisting content.

Fortunately, LangChain provides an indexing API to make it easy to keep your documents in sync with your vector store. The API utilizes a class (`RecordManager`) to keep track of document writes into the vector store. When indexing content, hashes are computed for each document and the following information is stored in `RecordManager`:

- The document hash (hash of both page content and metadata)
- Write time
- The source ID (each document should include information in its meta-data to determine the ultimate source of this document).

In addition, the indexing API provides cleanup modes to help you decide how to delete existing documents in the vector store. For example, If you've made changes to how documents are processed before insertion or if source documents have changed, you may want to remove any existing documents that come from the same source as the new documents being indexed. If some source documents have been deleted, you'll want to delete all existing documents in the vector store and replace them with the re-indexed documents.

The modes are as follows:

- `None` mode does not do any automatic cleanup, allowing the user to manually do cleanup of old content.
- `Incremental` and `full` modes delete previous versions of the content if the content of the source document or derived documents has changed.
- `Full` mode will additionally delete any documents not included in documents currently being indexed.

Here's an example of the use of the indexing API with Postgres database set up as a record manager:

Python

```
from langchain.indexes import SQLRecordManager, index
from langchain_postgres.vectorstores import PGVector
from langchain_openai import OpenAIEmbeddings
from langchain.docstore.document import Document

connection = "postgresql+psycopg://langchain:langchain@localhost:6024/langc
collection_name = "my_docs"
embeddings_model = OpenAIEmbeddings(model="text-embedding-3-small")
namespace = "my_docs_namespace"

vectorstore = PGVector(
    embeddings=embeddings_model,
    collection_name=collection_name,
```

```
    connection=connection,
    use_jsonb=True,
)

record_manager = SQLRecordManager(
    namespace,
    db_url="postgresql+psycopg://langchain:langchain@localhost:6024/langcha
)

# Create the schema if it doesn't exist
record_manager.create_schema()

# Create documents
docs = [
    Document(page_content='there are cats in the pond', metadata={
        "id": 1, "source": "cats.txt"}),
    Document(page_content='ducks are also found in the pond', metadata={
        "id": 2, "source": "ducks.txt"}),
]

# Index the documents
index_1 = index(
    docs,
    record_manager,
    vectorstore,
    cleanup="incremental",  # prevent duplicate documents
    source_id_key="source",  # use the source field as the source_id
)

print("Index attempt 1:", index_1)

# second time you attempt to index, it will not add the documents again
index_2 = index(
    docs,
    record_manager,
    vectorstore,
    cleanup="incremental",
    source_id_key="source",
)

print("Index attempt 2:", index_2)

# If we mutate a document, the new version will be written and all old
# versions sharing the same source will be deleted.

docs[0].page_content = "I just modified this document!"

index_3 = index(
    docs,
    record_manager,
    vectorstore,
    cleanup="incremental",
    source_id_key="source",
```

```
)

print("Index attempt 3:", index_3)
```

JavaScript

```
/**
1. Ensure docker is installed and running (https://docs.docker.com/get-dock
2. Run the following command to start the postgres container:

docker run \
  --name pgvector-container \
  -e POSTGRES_USER=langchain \
  -e POSTGRES_PASSWORD=langchain \
  -e POSTGRES_DB=langchain \
  -p 6024:5432 \
  -d pgvector/pgvector:pg16
3. Use the connection string below for the postgres container
*/

import { PostgresRecordManager } from '@langchain/community/indexes/postgre
import { index } from 'langchain/indexes';
import { OpenAIEmbeddings } from '@langchain/openai';
import { PGVectorStore } from '@langchain/community/vectorstores/pgvector'
import { v4 as uuidv4 } from 'uuid';

const tableName = 'test_langchain';
const connectionString =
  'postgresql://langchain:langchain@localhost:6024/langchain';
// Load the document, split it into chunks

const config = {
  postgresConnectionOptions: {
    connectionString,
  },
  tableName: tableName,
  columns: {
    idColumnName: 'id',
    vectorColumnName: 'vector',
    contentColumnName: 'content',
    metadataColumnName: 'metadata',
  },
};

const vectorStore = await PGVectorStore.initialize(
  new OpenAIEmbeddings(),
  config
);

// Create a new record manager
const recordManagerConfig = {
  postgresConnectionOptions: {
```

```
    connectionString,
  },
  tableName: 'upsertion_records',
};
const recordManager = new PostgresRecordManager(
  'test_namespace',
  recordManagerConfig
);

// Create the schema if it doesn't exist
await recordManager.createSchema();

const docs = [
  {
    pageContent: 'there are cats in the pond',
    metadata: { id: uuidv4(), source: 'cats.txt' },
  },
  {
    pageContent: 'ducks are also found in the pond',
    metadata: { id: uuidv4(), source: 'ducks.txt' },
  },
];

// the first attempt will index both documents
const index_attempt_1 = await index({
  docsSource: docs,
  recordManager,
  vectorStore,
  options: {
    // prevent duplicate documents by id from being indexed
    cleanup: 'incremental',
    // the key in the metadata that will be used to identify the document
    sourceIdKey: 'source',
  },
});

console.log(index_attempt_1);

// the second attempt will skip indexing because the identical documents
// already exist
const index_attempt_2 = await index({
  docsSource: docs,
  recordManager,
  vectorStore,
  options: {
    cleanup: 'incremental',
    sourceIdKey: 'source',
  },
});

console.log(index_attempt_2);

// If we mutate a document, the new version will be written and all old
```

```
// versions sharing the same source will be deleted.
docs[0].pageContent = 'I modified the first document content';
const index_attempt_3 = await index({
  docsSource: docs,
  recordManager,
  vectorStore,
  options: {
    cleanup: 'incremental',
    sourceIdKey: 'source',
  },
});

console.log(index_attempt_3);
```

First, you create a record manager, which keeps track of which documents have been indexed before. Then you use the `index` function to synchronize your vector store with the new list of documents. In this example, we're using the incremental mode, so any documents that have the same ID as previous ones will be replaced with the new version.

Indexing Optimization

A basic RAG indexing stage involves naive text splitting and embedding of chunks of a given document. However, this basic approach leads to inconsistent retrieval results and a relatively high occurrence of hallucinations, especially when the data source contains images and tables.

There are various strategies to enhance the accuracy and performance of the indexing stage. We will cover three of them in the next sections: MultiVectorRetriever, RAPTOR, and ColBERT.

MultiVectorRetriever

A document that contains a mixture of text and tables cannot be simply split by text into chunks and embedded as context: the entire table can be easily lost. To solve this problem, we can decouple documents that we want to use for answer synthesis, from a reference that we want to use for the retriever. Figure 2-5 illustrates how.

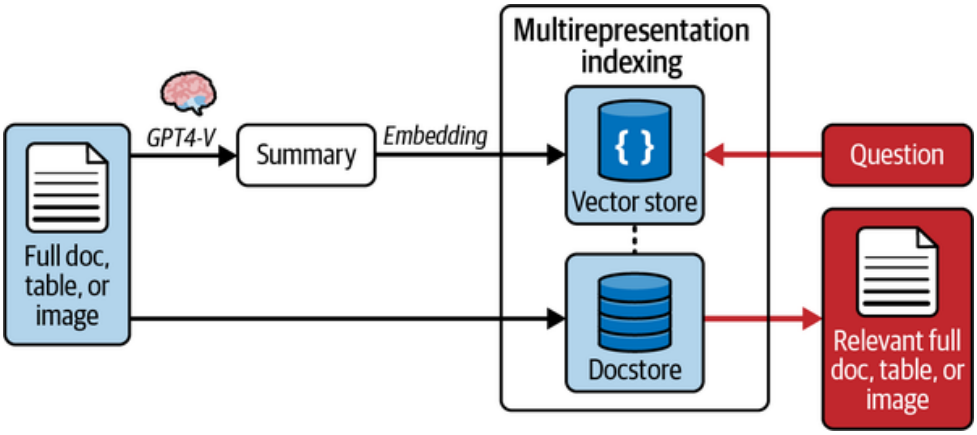

Figure 2-5. Indexing multiple representations of a single document

For example, in the case of a document that contains tables, we can first generate and embed summaries of table elements, ensuring each summary contains an `id` reference to the full raw table. Next, we store the raw referenced tables in a separate docstore. Finally, when a user's query retrieves a table summary, we pass the entire referenced raw table as context to the final prompt sent to the LLM for answer synthesis. This approach enables us to provide the model with the full context of information required to answer the question.

Here's an example. First, let's use the LLM to generate summaries of the documents:

Python

```
from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings
from langchain_postgres.vectorstores import PGVector
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
from pydantic import BaseModel
from langchain_core.runnables import RunnablePassthrough
from langchain_openai import ChatOpenAI
from langchain_core.documents import Document
from langchain.retrievers.multi_vector import MultiVectorRetriever
from langchain.storage import InMemoryStore
import uuid

connection = "postgresql+psycopg://langchain:langchain@localhost:6024/lang
collection_name = "summaries"
embeddings_model = OpenAIEmbeddings()
# Load the document
loader = TextLoader("./test.txt", encoding="utf-8")
docs = loader.load()

print("length of loaded docs: ", len(docs[0].page_content))
# Split the document
splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=2
```

```
chunks = splitter.split_documents(docs)

# The rest of your code remains the same, starting from:
prompt_text = "Summarize the following document:\n\n{doc}"

prompt = ChatPromptTemplate.from_template(prompt_text)
llm = ChatOpenAI(temperature=0, model="gpt-3.5-turbo")
summarize_chain = {
    "doc": lambda x: x.page_content} | prompt | llm | StrOutputParser()

# batch the chain across the chunks
summaries = summarize_chain.batch(chunks, {"max_concurrency": 5})
```

Next, let's define the vector store and docstore to store the raw summaries and their embeddings:

Python

```
# The vectorstore to use to index the child chunks
vectorstore = PGVector(
    embeddings=embeddings_model,
    collection_name=collection_name,
    connection=connection,
    use_jsonb=True,
)
# The storage layer for the parent documents
store = InMemoryStore()
id_key = "doc_id"

# indexing the summaries in our vector store, whilst retaining the original
# documents in our document store:
retriever = MultiVectorRetriever(
    vectorstore=vectorstore,
    docstore=store,
    id_key=id_key,
)

# Changed from summaries to chunks since we need same length as docs
doc_ids = [str(uuid.uuid4()) for _ in chunks]

# Each summary is linked to the original document by the doc_id
summary_docs = [
    Document(page_content=s, metadata={id_key: doc_ids[i]})
    for i, s in enumerate(summaries)
]

# Add the document summaries to the vector store for similarity search
retriever.vectorstore.add_documents(summary_docs)

# Store the original documents in the document store, linked to their summa
# via doc_ids
# This allows us to first search summaries efficiently, then fetch the full
```

```
# docs when needed
retriever.docstore.mset(list(zip(doc_ids, chunks)))

# vector store retrieves the summaries
sub_docs = retriever.vectorstore.similarity_search(
    "chapter on philosophy", k=2)
```

Finally, let's retrieve the relevant full context document based on a query:

Python

```
# Whereas the retriever will return the larger source document chunks:
retrieved_docs = retriever.invoke("chapter on philosophy")
```

Here's the full implementation in JavaScript:

JavaScript

```
import * as uuid from 'uuid';
import { MultiVectorRetriever } from 'langchain/retrievers/multi_vector';
import { OpenAIEmbeddings } from '@langchain/openai';
import { RecursiveCharacterTextSplitter } from '@langchain/textsplitters';
import { InMemoryStore } from '@langchain/core/stores';
import { TextLoader } from 'langchain/document_loaders/fs/text';
import { Document } from '@langchain/core/documents';
import { PGVectorStore } from '@langchain/community/vectorstores/pgvector'
import { ChatOpenAI } from '@langchain/openai';
import { PromptTemplate } from '@langchain/core/prompts';
import { RunnableSequence } from '@langchain/core/runnables';
import { StringOutputParser } from '@langchain/core/output_parsers';

const connectionString =
  'postgresql://langchain:langchain@localhost:6024/langchain';
const collectionName = 'summaries';

const textLoader = new TextLoader('./test.txt');
const parentDocuments = await textLoader.load();
const splitter = new RecursiveCharacterTextSplitter({
  chunkSize: 10000,
  chunkOverlap: 20,
});
const docs = await splitter.splitDocuments(parentDocuments);

const prompt = PromptTemplate.fromTemplate(
  `Summarize the following document:\n\n{doc}`
);

const llm = new ChatOpenAI({ modelName: 'gpt-3.5-turbo' });
```

```
const chain = RunnableSequence.from([
  { doc: (doc) => doc.pageContent },
  prompt,
  llm,
  new StringOutputParser(),
]);

// batch summarization chain across the chunks
const summaries = await chain.batch(docs, {
  maxConcurrency: 5,
});

const idKey = 'doc_id';
const docIds = docs.map((_) => uuid.v4());
// create summary docs with metadata linking to the original docs
const summaryDocs = summaries.map((summary, i) => {
  const summaryDoc = new Document({
    pageContent: summary,
    metadata: {
      [idKey]: docIds[i],
    },
  });
  return summaryDoc;
});

// The byteStore to use to store the original chunks
const byteStore = new InMemoryStore();

// vector store for the summaries
const vectorStore = await PGVectorStore.fromDocuments(
  docs,
  new OpenAIEmbeddings(),
  {
    postgresConnectionOptions: {
      connectionString,
    },
  }
);

const retriever = new MultiVectorRetriever({
  vectorstore: vectorStore,
  byteStore,
  idKey,
});

const keyValuePairs = docs.map((originalDoc, i) => [docIds[i], originalDoc

// Use the retriever to add the original chunks to the document store
await retriever.docstore.mset(keyValuePairs);

// Vectorstore alone retrieves the small chunks
const vectorstoreResult = await retriever.vectorstore.similaritySearch(
  'chapter on philosophy',
```

```
  2
);
console.log(`summary: ${vectorstoreResult[0].pageContent}`);
console.log(
  `summary retrieved length: ${vectorstoreResult[0].pageContent.length}`
);

// Retriever returns larger chunk result
const retrieverResult = await retriever.invoke('chapter on philosophy');
console.log(
  `multi-vector retrieved chunk length: ${retrieverResult[0].pageContent.le
);
```

RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval

RAG systems need to handle lower-level questions that reference specific facts found in a single document or higher-level questions that distill ideas that span many documents. Handling both types of questions can be a challenge with typical k-nearest neighbors (k-NN) retrieval over document chunks.

Recursive abstractive processing for tree-organized retrieval (RAPTOR) is an effective strategy that involves creating document summaries that capture higher-level concepts, embedding and clustering those documents, and then summarizing each cluster.[2] This is done recursively, producing a tree of summaries with increasingly high-level concepts. The summaries and initial documents are then indexed together, giving coverage across lower-to-higher-level user questions. Figure 2-6 illustrates.

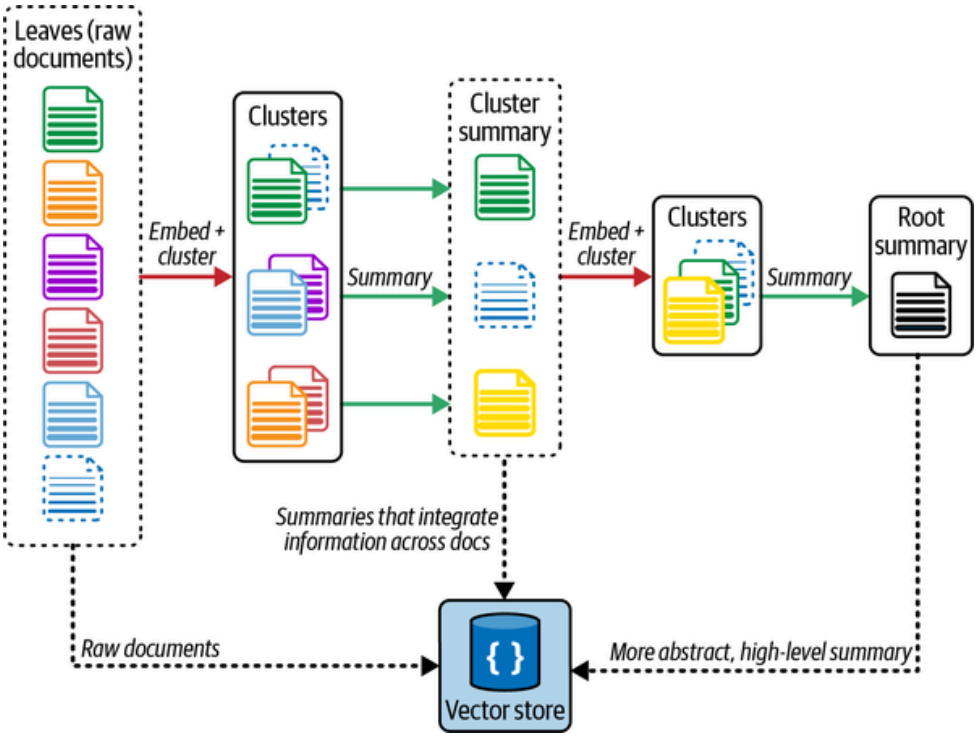

Figure 2-6. Recursively summarizing documents

ColBERT: Optimizing Embeddings

One of the challenges of using embeddings models during the indexing stage is that they compress text into fixed-length (vector) representations that capture the semantic content of the document. Although this compression is useful for retrieval, embedding irrelevant or redundant content may lead to hallucinations in the final LLM output.

One solution to this problem is to do the following:

1. Generate contextual embeddings for each token in the document and query.
2. Calculate and score similarity between each query token and all document tokens.
3. Sum the maximum similarity score of each query embedding to any of the document embeddings to get a score for each document.

This results in a granular and effective embedding approach for better retrieval. Fortunately, the embedding model called ColBERT embodies the solution to this problem.[3]

Here's how we can utilize ColBERT for optimal embedding of our data:

Python

```python
# RAGatouille is a library that makes it simple to use ColBERT
#! pip install -U ragatouille

from ragatouille import RAGPretrainedModel
RAG = RAGPretrainedModel.from_pretrained("colbert-ir/colbertv2.0")

import requests

def get_wikipedia_page(title: str):
    """
    Retrieve the full text content of a Wikipedia page.

    :param title: str - Title of the Wikipedia page.
    :return: str - Full text content of the page as raw string.
    """
    # Wikipedia API endpoint
    URL = "https://en.wikipedia.org/w/api.php"

    # Parameters for the API request
    params = {
        "action": "query",
        "format": "json",
        "titles": title,
        "prop": "extracts",
        "explaintext": True,
```

```
    }

    # Custom User-Agent header to comply with Wikipedia's best practices
    headers = {"User-Agent": "RAGatouille_tutorial/0.0.1"}

    response = requests.get(URL, params=params, headers=headers)
    data = response.json()

    # Extracting page content
    page = next(iter(data["query"]["pages"].values()))
    return page["extract"] if "extract" in page else None

full_document = get_wikipedia_page("Hayao_Miyazaki")

## Create an index
RAG.index(
    collection=[full_document],
    index_name="Miyazaki-123",
    max_document_length=180,
    split_documents=True,
)

#query
results = RAG.search(query="What animation studio did Miyazaki found?", k=
results

#utilize langchain retriever
retriever = RAG.as_langchain_retriever(k=3)
retriever.invoke("What animation studio did Miyazaki found?")
```

By using ColBERT, you can improve the relevancy of retrieved documents used as context by the LLM.

Summary

In this chapter, you've learned how to prepare and preprocess your documents for your LLM application using various LangChain's modules. The document loaders enable you to extract text from your data source, the text splitters help you split your document into semantically similar chunks, and the embeddings models convert your text into vector representations of their meaning.

Separately, vector stores allow you to perform CRUD operations on these embeddings alongside complex calculations to compute semantically similar chunks of text. Finally, indexing optimization strategies enable your AI app to improve the quality of embeddings and perform accurate retrieval of documents that contain semistructured data including tables.

In Chapter 3, you'll learn how to efficiently retrieve the most similar chunks of documents from your vector store based on your query, provide it as context the model can see, and then generate an accurate output.

1 Arvind Neelakantan et al., "Text and Code Embeddings by Contrastive Pre-Training", arXiv, January 21, 2022.

2 Parth Sarthi et al., "RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval", arXiv, January 31, 2024. Paper published at ICLR 2024.

3 Keshav Santhanam et al., "ColBERTv2: Effective and Efficient Retrieval via Lightweight Late Interaction", arXiv, December 2, 2021.